INFORME DE PROYECTO

Diseño e implementación de una CNC láser controlada mediante una placa NUCLEO-F446RE

Nicolas Patricelli[1]

Federico Barrios Retta[2]

[1]Facultad de Ingeniería, Universidad Nacional de Cuyo, Argentina
[2]Facultad de Ingeniería, Universidad Nacional de Cuyo, Argentina

Resumen

Se presenta el desarrollo de un controlador para una CNC de grabado y corte láser, basado en una placa NUCLEO-F446RE. Se describen la estructura mecánica adoptada, las interfaces eléctricas y el firmware modular en C. Las instrucciones generadas con LaserGRBL se reciben por UART, se interpretan y se convierten en consignas para el mecanismo CoreXY y el eje Z. La coordinación de los motores utiliza interpolación discreta y temporizadores, mientras que el accionamiento del láser se comanda mediante PWM. El documento corresponde a un avance del proyecto; la caracterización experimental y las conclusiones se incorporarán al completar los ensayos.

Índice

1. Introducción y objetivos **3**
1.1. Objetivo general . . . 3
1.2. Objetivos específicos . . . 3

2. Descripción General del Sistema **3**

3. Sistema Mecánico y Actuadores **3**
3.1. Armadura de la CNC . . . 3
3.2. Motores paso a paso . . . 4

4. Hardware e Interfaces Eléctricas **4**
4.1. Microcontrolador seleccionado . . . 4
4.2. Motores seleccionados . . . 4
4.3. Drivers seleccionados . . . 5
4.4. Conexiones eléctricas . . . 5

5. Arquitectura del Firmware **9**
5.1. Organización modular . . . 9
5.2. Inicialización y programa principal . . . 11
5.3. Coordinación de los módulos . . . 11
5.4. Ejecución mediante interrupciones . . . 11
5.5. Representación de los estados de operación . . . 11

6. Comunicación e Interpretación de G-Code **12**
6.1. Generación y envío de instrucciones mediante LaserGRBL . . . 12
6.2. Comunicación serie y tratamiento de mensajes . . . 12
6.3. Interpretación de las instrucciones . . . 13
6.4. Comandos reconocidos y estado modal . . . 13
6.5. Anticipación de segmentos mediante lookahead . . . 14
6.6. Alcance de la compatibilidad con LaserGRBL . . . 15

7. Generación y Coordinación del Movimiento **15**
7.1. Cinemática del mecanismo CoreXY . . . 15
7.2. Conversión de desplazamientos en pulsos . . . 16
7.3. Interpolación y sincronización de los motores . . . 16
7.4. Temporización y perfil de velocidad . . . 17
7.5. Ejecución del barrido de grabado . . . 18
7.6. Movimiento independiente del eje Z . . . 18
7.7. Resolución y exactitud del posicionamiento . . . 18

8. Referenciación, Límites y Control del Láser **18**

9. Ensayos y Resultados **18**

10. Limitaciones y Conclusiones **18**

1. Introducción y objetivos

El grabado mediante láser requiere que el desplazamiento relativo entre la herramienta y la pieza sea coordinado con el accionamiento del láser. Para ejecutar una trayectoria definida, deben ser interpretadas las instrucciones de trabajo y generadas las señales que gobiernan los actuadores de la máquina.

En el marco de la asignatura *Microcontroladores y Electrónica de Potencia*, se desarrolló el control de una CNC destinada al grabado y corte láser. El proyecto permite integrar los contenidos relacionados con la programación de sistemas embebidos, la comunicación serie, la utilización de sensores y el accionamiento de motores paso a paso.

1.1. Objetivo general

Diseñar e implementar un sistema de control basado en un microcontrolador para ejecutar trayectorias de grabado mediante láser a partir de instrucciones enviadas desde una computadora.

1.2. Objetivos específicos

Se plantean como objetivos específicos la interpretación de las instrucciones de trabajo, la coordinación de los desplazamientos en el plano de grabado, el ajuste de la distancia relativa entre el láser y la pieza, y la regulación del accionamiento del láser durante la ejecución de las trayectorias.

Asimismo, se busca establecer una referencia de posición mediante sensores de fin de carrera y verificar el funcionamiento del sistema mediante ensayos de movimiento y grabado.

2. Descripción General del Sistema

La máquina desarrollada consiste en un sistema de control numérico computarizado (CNC) destinado al grabado y al corte mediante láser. Durante estos procesos, el cabezal es desplazado respecto de la pieza siguiendo una trayectoria definida por instrucciones G-code, mientras que el accionamiento del láser es coordinado con el movimiento.

El posicionamiento del cabezal en el plano X–Y se realiza mediante un mecanismo de cinemática CoreXY, accionado por dos motores paso a paso. Se incorpora, además, un eje Z que permite desplazar el cabezal para ajustar la distancia entre el módulo láser y la superficie de trabajo. El eje Z es accionado de manera independiente de los desplazamientos coordinados en el plano X–Y.

Las trayectorias de trabajo son definidas mediante un archivo G-code generado en una computadora. Las instrucciones son transmitidas al microcontrolador STM32 de una placa NUCLEO-F446RE, donde son interpretadas para generar las señales de paso, dirección y habilitación de los drivers de los motores. Asimismo, se genera una señal de modulación por ancho de pulso (PWM) destinada a comandar el accionamiento del módulo láser.

El sistema incorpora finales de carrera para establecer una referencia de posición mediante un procedimiento de referenciación o *homing*. A partir de dicha referencia, los desplazamientos son determinados mediante las consignas y los pasos comandados a los motores. El posicionamiento se realiza en lazo abierto, sin medición continua de la posición del cabezal.

3. Sistema Mecánico y Actuadores

3.1. Armadura de la CNC

La armadura se construyó a partir del diseño *Core XY*, publicado por Javierarnedo en Thingiverse [1]. Se conservaron los perfiles y la configuración del bastidor. Las dimensiones exteriores y el recorrido útil deben relevarse en la máquina; los límites definidos en el firmware no equivalen a esas medidas.

Se utilizaron perfiles ranurados de aluminio de sección nominal $20 \times 20\,\mathrm{mm}$, dispuestos en dos marcos rectangulares unidos mediante cuatro montantes. Las uniones atornilladas y las escuadras permiten fijar los soportes de guiado y transmisión. La alineación del bastidor y la ausencia de holguras en las uniones condicionan la geometría del recorrido.

3.2. Motores paso a paso

El mecanismo se acciona mediante cuatro motores bipolares: dos para la transmisión CoreXY y dos para Z. Su identificación se presenta en la Subsección 4.2; la conversión de desplazamientos en pulsos se desarrolla en la Sección 7.

El posicionamiento es de lazo abierto. Los finales de carrera establecen una referencia, pero no proporcionan una medición continua del cabezal. La pérdida de pasos, las holguras y la deformación de las correas pueden producir diferencias entre la posición lógica y la física.

4. Hardware e Interfaces Eléctricas

4.1. Microcontrolador seleccionado

El sistema de control fue implementado mediante una placa de desarrollo NUCLEO-F446RE, que incorpora un microcontrolador STM32F446RET6 de la familia STM32F4. Este dispositivo dispone de un núcleo Arm Cortex-M4 de 32 bits, con unidad de punto flotante de precisión simple, 512 KB de memoria Flash y 128 KB de memoria SRAM [7].

La unidad de punto flotante proporciona soporte para las operaciones aritméticas empleadas en el procesamiento de coordenadas y en los cálculos de movimiento. Aunque el microcontrolador admite una frecuencia máxima de 180 MHz, en la configuración del proyecto se utiliza un reloj de sistema de 84 MHz.

Para la temporización del movimiento se utilizan los temporizadores TIM2 y TIM4, asociados a la generación de pasos de los motores del mecanismo CoreXY y del eje Z, respectivamente. El canal 1 del temporizador TIM3 se utiliza para generar la señal PWM que comanda el módulo láser.

La comunicación con la computadora se realiza mediante USART2, configurada a 115200 bit/s. La interfaz de puerto serie virtual de la placa permite establecer esta comunicación a través de la conexión USB del ST-LINK. Asimismo, este dispositivo integrado permite programar y depurar el microcontrolador durante el desarrollo.

Las señales de paso, dirección y habilitación de los drivers se generan mediante pines GPIO. También se utilizan entradas digitales para recibir las señales de los finales de carrera. Los conectores de expansión de la placa permiten acceder a los pines necesarios para estas interfaces.

La disponibilidad de temporizadores, comunicación serie, salidas PWM y pines de entrada y salida permite concentrar en un mismo microcontrolador la interpretación de instrucciones, la coordinación del movimiento y el comando de los actuadores.

4.2. Motores seleccionados

Para el accionamiento de la máquina se utilizaron cuatro motores paso a paso bipolares de tamaño de brida NEMA 17, marca Usongshine, modelo `17HS3401`.

Dos motores accionan el mecanismo CoreXY, correspondientes a los canales A y B. Los otros dos se utilizan para el desplazamiento del eje Z y se conectan en paralelo a un mismo driver.

Cada motor dispone de dos bobinas, cuyo accionamiento requiere invertir el sentido de la corriente para establecer la secuencia de excitación. Esta función es realizada por los módulos TB6600, mientras que el microcontrolador genera las señales de paso, dirección y habilitación.

La denominación NEMA 17 identifica el tamaño de la brida de montaje y no determina por sí sola la corriente nominal, el ángulo de paso ni el par disponible. Por ello, la caracterización de los actuadores debe completarse mediante la documentación correspondiente al modelo utilizado.

En el firmware se utiliza un ángulo de paso de $1{,}8°$, equivalente a 200 pasos completos por revolución, y una subdivisión de $1/16$ de paso. La conversión a pulsos por revolución y por milímetro se desarrolla en la Sección 7. Estos parámetros deben coincidir con el ángulo nominal del motor y con el ajuste de micropaso seleccionado en los drivers.

El ajuste de corriente de cada TB6600 debe establecerse a partir de la corriente nominal de los bobinados. Para el canal Z debe considerarse, además, la conexión de dos motores en paralelo, puesto que la corriente suministrada por el driver se distribuye entre los bobinados de ambos actuadores. El valor seleccionado y la configuración de los interruptores DIP quedan pendientes de documentar.

Cuadro 1. Identificación y características de los motores utilizados.

Parámetro	Identificación o valor
Marca	Usongshine
Modelo	17HS3401
Tamaño de brida	NEMA 17
Tipo de accionamiento	Bipolar, dos fases
Cantidad instalada	Cuatro motores
Lote relevado	202331120
Ángulo utilizado por el firmware	$1{,}8^\circ$
Corriente y par nominales	Pendientes de ficha del modelo

4.3. Drivers seleccionados

Para el accionamiento de los motores paso a paso se utilizaron módulos driver comercializados como TB6600. Su función consiste en recibir las señales de comando generadas por el microcontrolador y establecer la excitación de los bobinados necesaria para producir el movimiento.

Mediante estos módulos se separa la generación de las consignas de movimiento del suministro de energía a los motores. El microcontrolador determina la cantidad de pulsos, su frecuencia y el sentido de desplazamiento, mientras que el driver gobierna las corrientes de los bobinados a partir de su alimentación de potencia.

Interfaz de comando. El movimiento se comanda mediante las entradas PUL y DIR. Cada pulso válido recibido en PUL produce un avance en la secuencia de excitación, cuya magnitud depende de la subdivisión de pasos seleccionada. La entrada DIR establece el sentido de avance de dicha secuencia. Además, se dispone de una entrada ENA para controlar la habilitación del accionamiento.

Las entradas de comando se presentan mediante pares de terminales PUL+/PUL-, DIR+/DIR- y ENA+/ENA-. Según las especificaciones disponibles, estas entradas son optoacopladas y están destinadas a señales de 5 V, con una corriente de señal comprendida entre 8 y 15 mA. Por lo tanto, su conexionado debe considerar tanto la tensión aplicada como la corriente requerida por la interfaz de entrada.

Configuración de corriente y micropasos. El módulo permite seleccionar la corriente de salida mediante interruptores DIP. Este ajuste establece el nivel de corriente utilizado para la excitación de los bobinados y debe determinarse a partir de las características del motor conectado. La corriente máxima admitida por el driver no constituye, por sí sola, el valor que debe utilizarse en el proyecto.

También puede seleccionarse la subdivisión de pasos. En el funcionamiento por micropasos, las corrientes de los bobinados son modificadas para generar posiciones intermedias entre los pasos completos del motor. Esto permite incrementar la cantidad de pulsos requeridos por vuelta y reducir el incremento angular nominal asociado a cada pulso.

El firmware supone una subdivisión de $1/16$ de paso. La posición de los interruptores DIP debe corresponder a ese valor; el ajuste de corriente depende del motor y no se deduce del tamaño NEMA. Para Z, la corriente del driver se reparte entre los dos motores conectados en paralelo. La conversión de esta configuración a pulsos se presenta en la Sección 7.

Características eléctricas. En la Tabla 2 se resumen las especificaciones recopiladas para el módulo. Estos valores deben contrastarse con la documentación de la versión instalada. La tensión de alimentación, la corriente seleccionada y la posición de los interruptores DIP utilizados en la CNC se documentarán junto con el esquema de conexiones.

La frecuencia de PUL gobierna la velocidad de la secuencia de excitación. El límite de 20 kHz es un dato declarado para el módulo, no una velocidad experimental de la CNC; el par disponible también depende de la alimentación, la corriente y la velocidad.

4.4. Conexiones eléctricas

Las interconexiones entre la placa de desarrollo, los drivers de los motores, los finales de carrera y el módulo de accionamiento del láser se representan en la Figura 1. El esquema fue elaborado en KiCad e identifica los terminales de los módulos, las redes de alimentación y las señales de control utilizadas por el firmware.

Parámetro	Valor declarado
Tensión de alimentación de potencia	9–42 V CC
Intervalo de corriente de salida	0,7–4,0 A
Niveles de corriente seleccionables	8
Frecuencia máxima de pulsos de entrada	Hasta 20 kHz
Tensión de las señales de entrada	5 V
Corriente de las señales de entrada	8–15 mA
Pulsos por vuelta para un motor de $1{,}8^\circ$	200–6400
Aislamiento de las entradas	Optoacoplado
Protecciones declaradas	Sobrecorriente y sobretemperatura

Cuadro 2. Especificaciones recopiladas del módulo TB6600, pendientes de verificación documental. Debe confirmarse si los valores de corriente indicados corresponden a corriente pico o eficaz.

Distribución de la alimentación. Se dispone de una fuente con entrada de $220\,\mathrm{V}$ de corriente alterna y salidas de $24\,\mathrm{V}$, $12\,\mathrm{V}$ y $5\,\mathrm{V}$ de corriente continua. La alimentación de potencia de los drivers TB6600 se realiza mediante la salida de $12\,\mathrm{V}$, con los tres módulos conectados en paralelo. La placa NUCLEO-F446RE se alimenta mediante [completar: USB o alimentación externa]. La salida de $5\,\mathrm{V}$ y el modo de alimentación de la NUCLEO quedan pendientes de confirmar. El esquema conecta la alimentación del módulo de accionamiento del láser a la red de 12 V; falta contrastar esa conexión con la identificación del módulo instalado.

En el esquema se distinguen las alimentaciones de potencia, las tensiones utilizadas por las interfaces de control y sus correspondientes retornos. Las uniones entre referencias eléctricas se representan según el cableado del sistema.

Interfaz con los drivers de los motores. El accionamiento de los motores es comandado mediante señales de paso, dirección y habilitación. Las señales de paso y dirección son independientes para los canales A, B y Z, mientras que la habilitación es compartida por los tres drivers.

El cableado documentado en el firmware corresponde a una conexión de ánodo común: los terminales `PUL+`, `DIR+` y `ENA+` se conectan a $3{,}3\,\mathrm{V}$, y los terminales negativos reciben las señales de los GPIO. La correspondencia entre señales y pines se presenta en la Tabla 3. La conexión a 3,3 V documentada en el código debe contrastarse con los 5 V declarados para las entradas del módulo. La habilitación de tres entradas en paralelo también requiere comprobar la corriente conjunta sobre PC7; el esquema no constituye una validación eléctrica de esa carga.

Conexión de los motores. Cada motor bipolar dispone de dos bobinas. Los terminales `A+` y `A−` del driver se conectan a los extremos de una bobina, y los terminales `B+` y `B−`, a los extremos de la otra.

Los motores CoreXY se identifican como M1 (A) y M2 (B) en el esquema. Cada uno dispone de un driver; estas letras deben distinguirse de A+/A- y B+/B-, que identifican las bobinas. Para el eje Z, el firmware documenta dos motores conectados en paralelo a un mismo driver. El esquema muestra las ramificaciones de cada salida hacia M3 y M4, correspondientes a los dos motores de Z.

Interfaz con los finales de carrera. Se utilizan tres señales de límite, correspondientes a X, Y y Z, conectadas a PB0, PB1 y PB2, respectivamente. Los módulos de final de carrera se representan mediante sus terminales de alimentación, masa y señal, conservando la identificación del conector utilizado.

El firmware configura estas entradas con resistencias internas de polarización a masa e interpreta un nivel bajo como la activación del límite. Por lo tanto, debe comprobarse que la salida de los módulos presente el nivel de reposo y de activación esperado por el programa. El esquema identifica alimentación de 3,3 V, masa y señal. La identificación exacta del módulo y el orden de los terminales deben comprobarse en el montaje.

Interfaz con el accionamiento del láser. La señal de control del láser se genera mediante el canal 1 de TIM3 y se obtiene en PC6, disponible en el terminal 4 del conector CN10 de la NUCLEO-F446RE. Esta red se identifica como `LASER_PWM`.

Con la configuración del temporizador utilizada en el firmware, la frecuencia nominal de la señal es de $1\,\mathrm{kHz}$. Su ciclo de trabajo es modificado para establecer la consigna de accionamiento del módulo láser. La red PC6 llega al módulo de

accionamiento y no directamente al diodo láser. El firmware identifica la función de control; la tensión admitida por esa entrada y la correspondencia del símbolo P-DA-01 con el módulo instalado quedan pendientes de verificar.

Cuadro 3. Asignación de señales de control y entradas de límite en la NUCLEO-F446RE.

Señal	GPIO	Destino o procedencia
STEP_A	PC0	PUL- del driver A
DIR_A	PC1	DIR- del driver A
STEP_B	PC2	PUL- del driver B
DIR_B	PC3	DIR- del driver B
STEP_Z	PC4	PUL- del driver Z
DIR_Z	PC5	DIR- del driver Z
MOTOR_EN	PC7	ENA- de los tres drivers
LIMIT_X	PB0	Señal del final de carrera X
LIMIT_Y	PB1	Señal del final de carrera Y
LIMIT_Z	PB2	Señal del final de carrera Z
LASER_PWM	PC6	Entrada de control del módulo láser

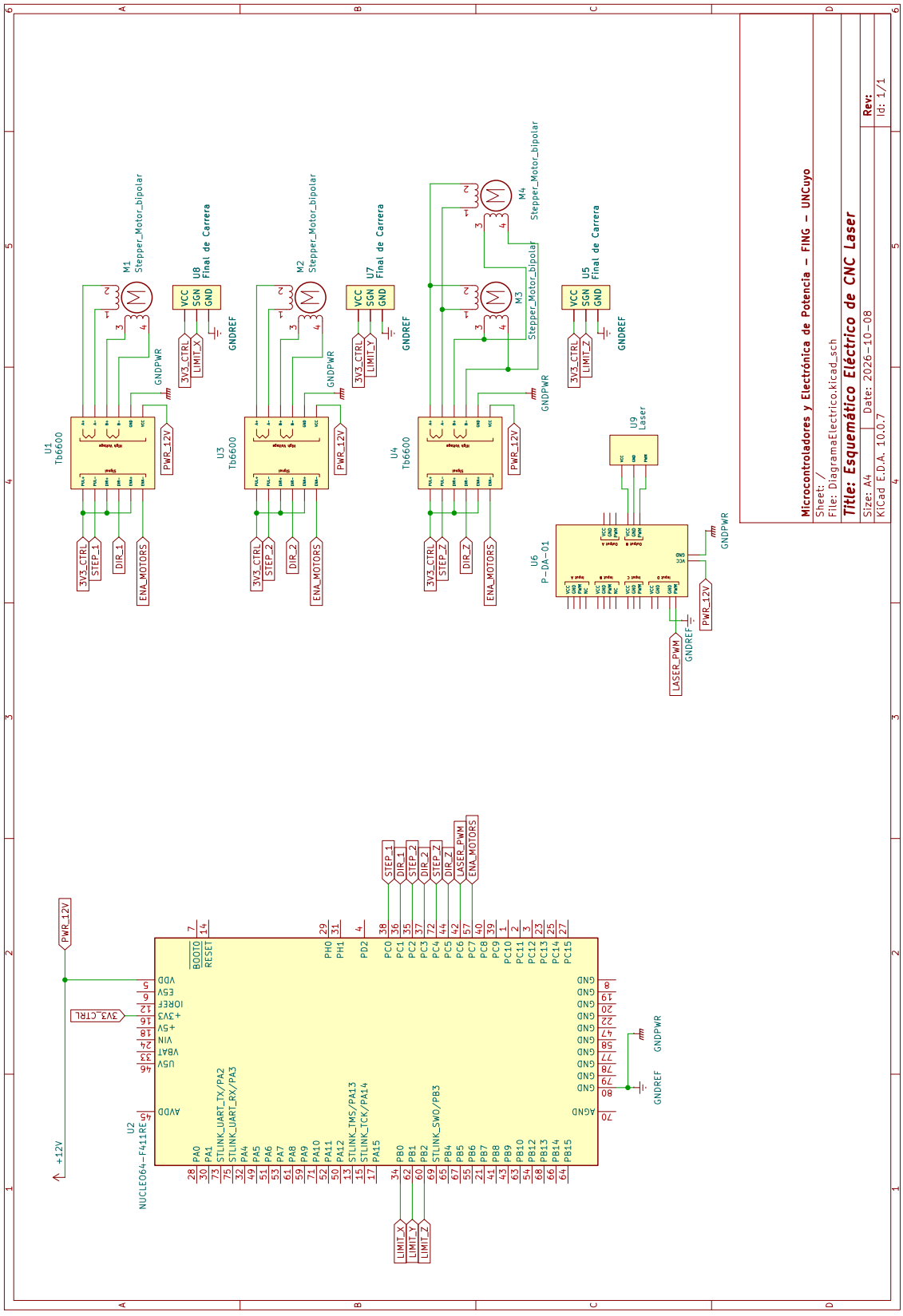


Figura 1. Esquema de interconexión elaborado en KiCad. Se conserva el original: el símbolo de la placa está rotulado F411RE, mientras que la placa y el firmware del proyecto corresponden a F446RE. Debe corregirse esa identificación en KiCad y verificar los conectores del montaje.

5. Arquitectura del Firmware

5.1. Organización modular

El firmware fue desarrollado en lenguaje C mediante una organización modular. La descripción se contrasta con el commit `688d31a` del repositorio del proyecto [5]. Las responsabilidades del sistema se distribuyen entre módulos de interpretación de instrucciones, generación de movimiento, referenciación, control del láser y habilitación de los drivers.

Los módulos funcionales se organizan mediante archivos de encabezado, en los que se declara su interfaz pública, y archivos de implementación, en los que se definen las operaciones correspondientes. Los encabezados de configuración reúnen constantes y asignaciones de pines, sin requerir necesariamente un archivo de implementación.

El estado interno y las funciones auxiliares de los módulos se mantienen, cuando corresponde, mediante declaraciones con el especificador `static`. En las declaraciones a nivel de archivo, este especificador establece enlace interno y evita que sus identificadores sean utilizados directamente desde otras unidades de compilación. De esta manera, el acceso al módulo se realiza mediante las funciones declaradas en su encabezado.

Las variables locales declaradas con `static` conservan su valor entre llamadas sucesivas. Este recurso es utilizado, por ejemplo, para recordar las condiciones de continuidad entre segmentos de movimiento. El ocultamiento del estado interno permite modificar su representación sin alterar la interfaz utilizada por los demás módulos.

Las dependencias funcionales se muestran en la Figura 2. Se utilizan bloques de módulos C, no clases de una implementación orientada a objetos. El diagrama UML elaborado durante el desarrollo se conserva como antecedente; las interfaces actuales incluyen también el watchdog y la regulación dinámica del láser.

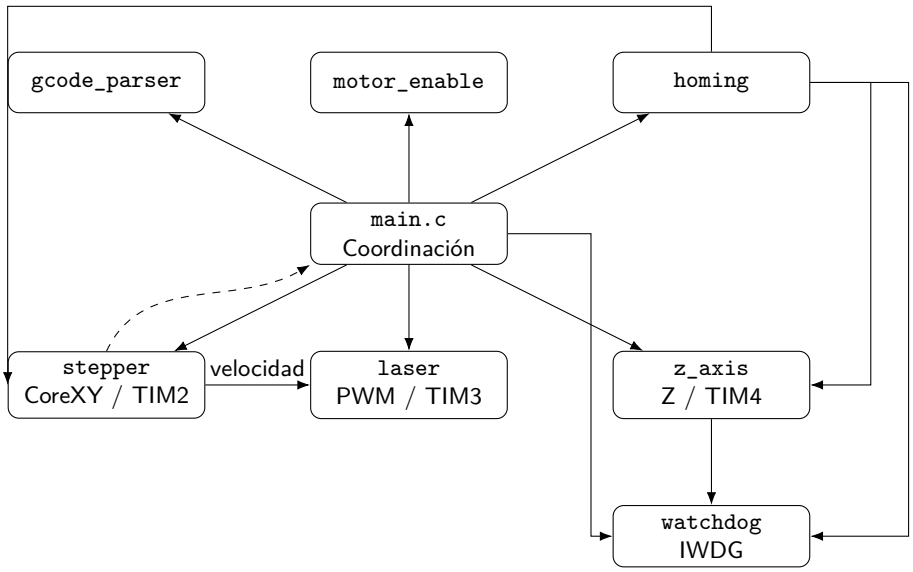


Figura 2. Dependencias de ejecución entre módulos. Las flechas parten del módulo que utiliza la interfaz; la línea discontinua representa la callback de servicio registrada por `main.c`.

La Figura 3 muestra la estructura de la copia de trabajo. El repositorio remoto sitúa `Core/` y `Drivers/` en la raíz; la carpeta local `Code/` los agrupa sin alterar sus responsabilidades.

Coordinación de la aplicación: `main.c`. En este archivo se inicializan los periféricos y los módulos funcionales, y se implementa el bucle principal. También se construyen las líneas recibidas, se atienden los comandos especiales y se determinan las acciones correspondientes a las instrucciones G-code. Se conservan el modo de coordenadas, el avance modal, el segmento X–Y pendiente y las banderas asociadas a la comunicación y a la alarma.

Interpretación de instrucciones: `gcode_parser`. Este módulo transforma una línea de texto en una estructura `GcodeCommand`. En ella se identifican las coordenadas, el avance, el parámetro de accionamiento del láser y los comandos reconocidos. El módulo realiza la interpretación de la instrucción; la decisión sobre su ejecución y la conservación del estado modal corresponden a la aplicación principal.

```
Code/
|-- CNC_laser.ioc
|-- Core/
|   |-- Inc/
|   |   |-- main.h, corexy_config.h
|   |   |-- gcode_parser.h, homing.h
|   |   |-- stepper.h, z_axis.h
|   |   |-- laser.h, motor_enable.h, watchdog.h
|   |   `-- ... encabezados de soporte
|   |-- Src/
|   |   |-- main.c, gcode_parser.c
|   |   |-- stepper.c, z_axis.c, homing.c
|   |   |-- laser.c, motor_enable.c, watchdog.c
|   |   |-- stm32f4xx_it.c, stm32f4xx_hal_msp.c
|   |   `-- ... archivos de soporte
|   `-- Startup/
|-- Drivers/
|   |-- CMSIS/
|   `-- STM32F4xx_HAL_Driver/
|-- STM32F446RETX_FLASH.ld
`-- STM32F446RETX_RAM.ld
```

Figura 3. Estructura seleccionada del proyecto de firmware. Se agrupan nombres en algunas filas y se abrevian los archivos de soporte y las bibliotecas.

Movimiento CoreXY: `stepper`. Se implementa la transformación de las consignas X–Y en desplazamientos de los motores A y B, junto con la coordinación de los pasos y el perfil de velocidad. El estado del movimiento y las posiciones lógicas se conservan internamente. La generación de pulsos es atendida mediante interrupciones de TIM2, mientras que la interfaz permite solicitar desplazamientos, consultar o establecer la posición lógica y detener el movimiento.

Movimiento del eje Z: `z_axis`. Se implementa el accionamiento del eje Z mediante consignas de posición y avance. El módulo conserva la posición lógica y el estado del desplazamiento, y actualiza los pasos mediante interrupciones de TIM4. Su ejecución es independiente del mecanismo de almacenamiento y continuidad de segmentos X–Y.

Referenciación: `homing`. Se implementa la secuencia de búsqueda de los finales de carrera en el orden Z, X e Y. La maniobra comprende una aproximación inicial, un retroceso, una segunda aproximación a menor velocidad y una separación final del contacto. Se utilizan las funciones de movimiento de los módulos CoreXY y Z, y se mantiene una bandera que identifica la ejecución de la referenciación.

Accionamiento del láser: `laser`. Se administra la habilitación del láser y la actualización del ciclo de trabajo de la señal PWM generada por TIM3. El parámetro S se limita al intervalo configurado y se convierte en un valor de comparación del temporizador. Se distingue entre la deshabilitación del láser y la anulación de su salida durante un movimiento rápido. En modo M4, `stepper` actualiza la relación entre la velocidad comandada y la de crucero para ajustar el PWM; no se mide la velocidad física.

Habilitación de los drivers: `motor_enable`. Se administra la señal común de habilitación de los tres drivers y se conserva su estado lógico. La interfaz permite habilitar, deshabilitar y consultar dicho estado, y es utilizada por la aplicación para atender los comandos M17 y M84.

Supervisión: `watchdog`. El módulo consulta la causa del reinicio y refresca IWDG. Se utiliza desde el servicio principal, las esperas de Z y los retardos del homing. Un reinicio causado por watchdog activa una alarma al arrancar; no restaura la posición física anterior.

Configuración del movimiento: `corexy_config.h`. Este encabezado reúne las asignaciones de pines y los parámetros de conversión entre desplazamiento y pasos de los motores CoreXY, junto con constantes del perfil de velocidad y límites del área de trabajo. Los parámetros correspondientes al eje Z se encuentran definidos en `z_axis.h`.

5.2. Inicialización y programa principal

Al comenzar la ejecución se inicializa la biblioteca HAL, se configura el reloj del sistema y se establecen los parámetros de los GPIO, USART2 y los temporizadores TIM2, TIM3 y TIM4. También se inicializa el watchdog independiente IWDG.

Posteriormente se inicializan los módulos de referenciación, habilitación de motores, movimiento del eje Z, movimiento CoreXY y control del láser. También se inicia la recepción de datos por interrupción y se transmite el mensaje de bienvenida. Si el reinicio anterior fue provocado por el watchdog, se activa la alarma y se informa la pérdida de referencia de posición. La limpieza mediante `$X` no sustituye una nueva referenciación.

Una vez completada la inicialización, el programa permanece en un lazo principal. En él se atienden los mensajes pendientes, se verifica la condición de ejecución del movimiento X–Y almacenado y se extraen los caracteres del buffer de recepción para construir líneas completas.

Cada línea recibida es procesada mediante la función `process_gcode_line()`. Los comandos especiales son atendidos directamente, mientras que las instrucciones G-code son analizadas por el módulo `gcode_parser`. La estructura obtenida es utilizada para determinar las acciones sobre el movimiento, el láser y la habilitación de los motores.

5.3. Coordinación de los módulos

`main.c` mantiene los modos y el segmento pendiente, y solicita la ejecución mediante `flush_pending_move()`. Esta función fija el estado del láser para el segmento y llama a `stepper_move_to()`. El tratamiento de instrucciones y la anticipación se desarrollan en la Sección 6; la conversión CoreXY y la temporización, en la Sección 7.

Las consignas Z se ejecutan mediante `z_axis_move_to()`. El módulo `homing` utiliza operaciones de CoreXY y Z para referenciar la máquina, mientras que `motor_enable` administra la señal común de los drivers.

5.4. Ejecución mediante interrupciones

Las interrupciones de USART2 reciben los caracteres; las de TIM2 y TIM4 ejecutan los pasos de CoreXY y Z. TIM3 genera el PWM mediante hardware. Las funciones de movimiento esperan a que termine el segmento, mientras las interrupciones habilitadas siguen atendiendo los periféricos.

Durante la espera X–Y, una callback registrada por `main.c` atiende mensajes y refresca el watchdog. La espera Z y los retardos de homing refrescan el watchdog, pero no invocan ese mismo servicio de mensajes. El procesamiento de nuevas líneas sigue siendo secuencial. Los buffers y la respuesta al programa de computadora se describen en la Sección 6.

Las variables compartidas que lo requieren se declaran `volatile`. Este calificador expresa accesos que pueden cambiar fuera del flujo observado por el compilador; no sustituye la sincronización ni convierte una secuencia de operaciones en atómica. Su función es distinta del enlace interno establecido por `static`.

5.5. Representación de los estados de operación

La aplicación conserva su estado mediante banderas y estructuras distribuidas, sin una variable global que seleccione todos los modos. El movimiento activo, el segmento pendiente y la alarma representan condiciones distintas. La habilitación del láser, la de motores y el modo de coordenadas pueden coexistir con ellas.

La Figura 4 resume el flujo de llamadas del programa principal. Las rutinas de movimiento son bloqueantes y retornan al punto que las invocó; una interrupción de límite modifica banderas y solicita la detención, pero no elimina por sí sola el resto del procesamiento de una línea ya iniciada.

Cuando se confirma un límite fuera del homing, la callback detiene los movimientos, deshabilita el láser y los motores, invalida el pendiente y activa `alarm_active`. Al comenzar a procesar una nueva línea, se rechazan las instrucciones distintas de `$X`. También puede activarse la alarma al arrancar después de un reinicio por watchdog. Un mensaje de fallo de homing no establece esa bandera en la versión revisada.

Este bloqueo no se comprueba en todos los puntos del flujo. Tras un retorno por interrupción, una línea ya iniciada puede volver a almacenar un pendiente; además, el timeout no comprueba la alarma. La figura describe las llamadas efectivas y no supone un estado exclusivo de detención que el código no garantiza.

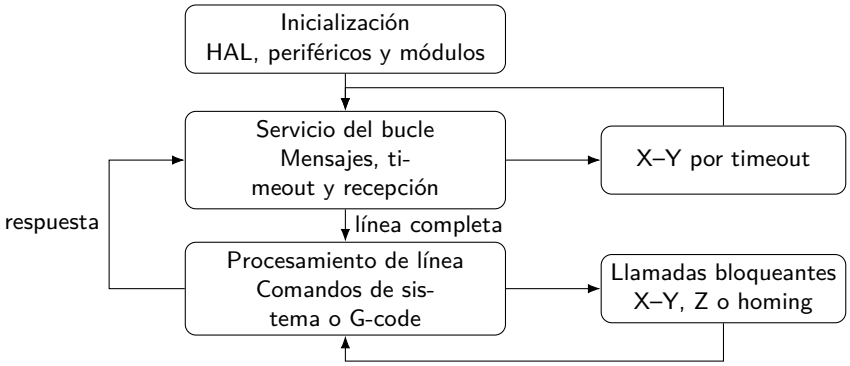


Figura 4. Flujo funcional reconstruido del firmware. Las ramas de X–Y distinguen el retorno al bucle del retorno al procesamiento de la línea.

6. Comunicación e Interpretación de G-Code

6.1. Generación y envío de instrucciones mediante LaserGRBL

La preparación de los trabajos y el envío de instrucciones a la máquina se realizan mediante LaserGRBL. Este programa permite importar imágenes, establecer parámetros del trabajo y generar una secuencia de instrucciones G-code. También proporciona una interfaz para transmitirlas al controlador y recibir sus respuestas [3].

La utilización de una herramienta existente permite delimitar el desarrollo al sistema embebido encargado de interpretar las consignas y accionar la máquina. La implementación de los algoritmos de procesamiento de imágenes y generación de trayectorias no forma parte del alcance del proyecto. Sin embargo, las instrucciones generadas deben pertenecer al conjunto reconocido por el firmware y utilizar parámetros compatibles con la configuración de la CNC.

En los trabajos de grabado de imágenes pueden generarse secuencias de segmentos lineales con diferentes valores de accionamiento del láser. La ejecución de estas trayectorias requiere conservar el orden de las instrucciones y asociar cada segmento con su consigna de velocidad y de accionamiento. La coordinación mecánica del barrido se desarrolla en la sección de generación de movimiento.

6.2. Comunicación serie y tratamiento de mensajes

La comunicación con la computadora se establece mediante USART2, configurada a 115200 bit/s, con ocho bits de datos, un bit de parada y sin paridad. La conexión USB del ST-LINK permite acceder a esta interfaz mediante un puerto serie virtual.

La recepción se realiza por interrupción, de a un carácter. Los caracteres ordinarios son almacenados en un buffer circular y posteriormente extraídos por el programa principal. Al encontrarse un retorno de carro o un salto de línea, se termina la cadena y se procesa la instrucción construida. Las líneas vacías no son enviadas al parser.

El buffer circular dispone de 256 posiciones, de las cuales 255 pueden contener caracteres pendientes. El buffer de construcción de líneas dispone de 96 posiciones, incluyendo el terminador de cadena. Estos límites condicionan la cantidad de información que puede conservarse mientras el programa principal se encuentra ocupado. Si el buffer circular se llena, se descartan los caracteres nuevos; si una línea supera 95 caracteres, su parte restante no se conserva. No se genera un error específico por estas condiciones.

Para mantener el intercambio con LaserGRBL se utilizan respuestas y mensajes con formato compatible con su interfaz [6]: `ok`, `error:n`, `ALARM:n` y `[MSG:...]`. Las instrucciones que alcanzan el final del procesamiento reciben una respuesta `ok`; determinados errores producen una respuesta `error:n` y un retorno anticipado.

La respuesta `ok` no debe interpretarse como una confirmación de que todos los desplazamientos asociados ya finalizaron. En particular, una consigna X–Y puede haber quedado almacenada como movimiento pendiente cuando se transmite esta respuesta.

Los caracteres `?` y `0x18` son tratados por la rutina de recepción sin incorporarse a las líneas G-code. El primero solicita un reporte de estado; el segundo solicita el mensaje de bienvenida y el restablecimiento de los offsets de trabajo. Las

acciones correspondientes se atienden mediante banderas y la función `service_realtime()`, desde el contexto del programa principal.

Durante los movimientos X–Y, esta función también es invocada desde la espera de la rutina de movimiento. De esta manera pueden atenderse solicitudes de estado sin comenzar el procesamiento de otra línea G-code.

6.3. Interpretación de las instrucciones

Las instrucciones recibidas son cadenas de caracteres, mientras que los módulos de control requieren valores numéricos y acciones identificadas. Por ello se utiliza un parser, es decir, una función que reconoce los elementos de la instrucción y los transforma en una representación que puede ser utilizada por el programa.

El análisis se implementa en `gcode_parser.c` mediante `gcode_parse_line()`. Su resultado se almacena en una estructura `GcodeCommand`, que contiene el tipo de movimiento, los parámetros numéricos y banderas que indican cuáles estaban presentes en la línea.

Cada palabra se identifica mediante una letra y un valor numérico. Por ejemplo, en `G1X12.5Y8F1200S300`, la letra G identifica el movimiento, X e Y indican coordenadas, F establece el avance y S representa la consigna de accionamiento del láser. Las banderas permiten distinguir un parámetro omitido de uno cuyo valor es explícitamente cero.

El recorrido de la cadena admite letras mayúsculas o minúsculas y palabras consecutivas sin espacios. Se descartan los comentarios iniciados por punto y coma y se omiten los comentarios entre paréntesis en las líneas procesadas. Las líneas que comienzan, después de los espacios iniciales, con un comentario son descartadas.

La Figura 5 distingue recepción, análisis y ejecución. La conversión numérica se realiza mediante `parse_gcode_number()`, que reconoce un signo opcional, dígitos y una parte decimal opcional. Esta función evita interpretar la secuencia `0X10` de una instrucción `G0X10` como un número hexadecimal. Durante las pruebas se detectó que la conversión de propósito general consumía la X y su valor como parte del número, en lugar de reconocer una nueva palabra. El lector decimal corrige ese caso; no admite notación científica y no sustituye una validación sintáctica completa.

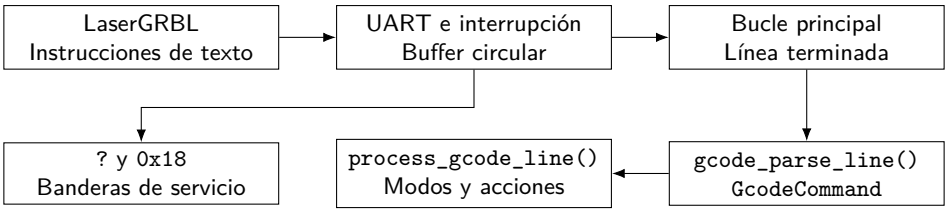


Figura 5. Recorrido de las instrucciones. Los comandos de sistema se resuelven en `main.c`; los caracteres de servicio no ingresan al parser de líneas.

El parser no ejecuta movimientos ni modifica directamente las salidas. La estructura obtenida es entregada a `process_gcode_line()`, en `main.c`, donde se aplica el estado modal, se calculan los destinos y se invocan las operaciones correspondientes.

6.4. Comandos reconocidos y estado modal

La implementación reconoce un subconjunto de instrucciones, seleccionado para las funciones requeridas por el proyecto. Este conjunto se resume en la Tabla 4.

Los comandos de sistema se atienden directamente en `main.c`. Entre ellos se encuentran `$H`, para solicitar la referenciación; `$X`, para limpiar la condición de alarma; `$J=`, para solicitar movimientos manuales; y `$$`, para consultar la configuración que el controlador informa al programa de computadora. LaserGRBL y GRBL son componentes distintos: el primero genera y envía instrucciones desde la computadora; en la NUCLEO se ejecuta el firmware propio [2].

El modo de coordenadas y el avance programado se conservan entre instrucciones. En los movimientos manuales introducidos mediante `$J=`, la selección de coordenadas absolutas o relativas se aplica a esa instrucción sin modificar el modo del programa.

Los offsets de trabajo permiten diferenciar las coordenadas utilizadas por el trabajo de las posiciones lógicas de máquina. En las consignas absolutas X–Y, los offsets se suman a las coordenadas recibidas para obtener el destino utilizado por el controlador.

Cuadro 4. Instrucciones y parámetros reconocidos por el firmware.

Instrucción	Función
`G0`	Traslado con salida del láser anulada
`G1`	Movimiento lineal con avance programado
`G90`, `G91`	Coordenadas absolutas o relativas
`G92`, `G92.1`	Establecer o borrar offsets X–Y
`X`, `Y`, `Z`	Consignas de posición
`F`	Avance expresado en mm/min
`S`	Consigna de accionamiento del láser
`M3`, `M4`	Habilitación constante o dinámica
`M5`	Deshabilitación del láser
`M17`, `M84`	Habilitación o liberación de motores

Las instrucciones desconocidas detectadas por el parser se identifican mediante la bandera `has_unknown`. En ese caso se rechaza la línea completa y se transmite una respuesta de error. Este tratamiento no constituye una implementación completa del lenguaje G-code ni una validación exhaustiva de todas sus combinaciones. Los códigos G y M se convierten parcialmente a enteros y algunas secuencias mal formadas pueden ser aceptadas. Los comandos `$H` y `$X` requieren, además, el formato literal previsto por `main.c`.

6.5. Anticipación de segmentos mediante lookahead

Para reducir las detenciones entre segmentos consecutivos se implementa una anticipación de un segmento, denominada *lookahead*. Una consigna X–Y válida es almacenada en una estructura `PendingMove`, que conserva el destino, el avance, la dirección normalizada y la información asociada al láser.

Cuando se recibe el siguiente destino X–Y válido, se compara su dirección con la del segmento pendiente. La comparación se realiza mediante el producto escalar de ambos vectores normalizados:

$$c = \hat{\mathbf{d}}_{\text{pendiente}} \cdot \hat{\mathbf{d}}_{\text{nuevo}}.$$

Si se cumple $c > 0{,}999$, los segmentos se consideran suficientemente alineados y con el mismo sentido. Esta condición es utilizada para solicitar que el segmento pendiente finalice a velocidad de crucero. En caso contrario, se solicita su desaceleración final.

El segmento pendiente es ejecutado mediante `flush_pending_move()`, y el nuevo segmento queda almacenado para la comparación posterior. Al calcular una nueva consigna relativa, se considera como referencia el destino del pendiente, de modo que las instrucciones mantienen su secuencia lógica.

Si no se recibe una continuación, el bucle principal ejecuta el pendiente al comprobar que transcurrieron más de 80 ms desde su almacenamiento. Esta condición es evaluada por software y no corresponde a una interrupción que se produzca exactamente al cumplirse dicho tiempo.

También se ejecuta el pendiente antes de determinadas operaciones que requieren conservar el orden, como un movimiento Z, M5, M84, un cambio de offsets o una actualización de S sin coordenadas X–Y.

La información del láser asociada al segmento es aplicada al comenzar su ejecución. De esta manera, la recepción de una consigna posterior no adelanta el valor S del nuevo segmento. El criterio de continuidad compara direcciones, pero no verifica igualdad de avances; un cambio de F entre segmentos alineados puede introducir una discontinuidad de velocidad.

Cuadro 5. Ejemplo de orden de ejecución del lookahead; no representa una medición de tiempos.

Evento	Acción	Pendiente al finalizar
Llega segmento 1	Se almacena; todavía no se mueve	Segmento 1
Llega segmento 2	Se compara la dirección y se ejecuta 1	Segmento 2
Llega segmento 3	Se compara la dirección y se ejecuta 2	Segmento 3
Sin continuación	Timeout mayor que 80 ms: se ejecuta 3	Ninguno

La Tabla 5 muestra que el último segmento requiere una condición de vaciado. Su valor S se aplica al ejecutar ese segmento, no al recibir el siguiente.

Este mecanismo conserva un solo segmento pendiente y utiliza un criterio geométrico de continuidad. No constituye un planificador de múltiples bloques con optimización general de velocidades.

6.6. Alcance de la compatibilidad con LaserGRBL

El firmware implementa las funciones de comunicación y los comandos necesarios para el uso previsto con LaserGRBL. La consulta $$ informa los valores $30, $31 y $32, asociados al intervalo de S y a la identificación del modo láser.

La consigna máxima de S está configurada en 1000. Este valor debe ser coherente con la configuración utilizada al generar el trabajo. Las instrucciones M3 y M4 seleccionan accionamiento constante y dinámico, respectivamente; el comportamiento del PWM se desarrolla en la sección de control del láser.

La compatibilidad se limita a las funciones implementadas. La transmisión de un mensaje de bienvenida con formato GRBL no implica disponer de todas las instrucciones, configuraciones o mecanismos de planificación del firmware original. Por ello, el G-code utilizado debe verificarse respecto del conjunto admitido. Por ejemplo, G21, G2 y G3 no se reconocen en esta versión. El código conserva `g_last_motion`, pero no lo utiliza para determinar si una línea sin G explícita es rápida; por ello, no se garantiza la herencia completa de G0.

El reporte de estado utiliza `Idle` o `Alarm`, conserva Z y los campos de avance y accionamiento en cero, e informa los offsets X–Y mediante `WCO`. No constituye una medición del estado físico ni un reporte completo de GRBL.

7. Generación y Coordinación del Movimiento

7.1. Cinemática del mecanismo CoreXY

El desplazamiento del cabezal en el plano X–Y se realiza mediante un mecanismo CoreXY. En esta configuración, los motores A y B actúan sobre una transmisión de correas que acopla sus movimientos con las dos coordenadas cartesianas. Por lo tanto, ninguno de los motores corresponde exclusivamente al eje X o al eje Y.

La utilización de esta transformación en el firmware responde a la configuración mecánica adoptada. Las instrucciones G-code establecen destinos cartesianos, mientras que los actuadores requieren consignas de giro. En consecuencia, cada desplazamiento solicitado debe ser convertido en los movimientos correspondientes a los motores A y B.

Con la convención de signos utilizada en el proyecto, la transformación cinemática inversa se expresa como [4]:

$$\Delta a = \Delta x + \Delta y, \qquad \Delta b = \Delta x - \Delta y, \tag{1}$$

donde Δx y Δy son los desplazamientos cartesianos del cabezal, y Δa y Δb representan los desplazamientos equivalentes de la transmisión asociados a los motores. Todas estas magnitudes se expresan en unidades de longitud.

La transformación cinemática directa permite obtener el desplazamiento cartesiano a partir de la actuación de ambos motores:

$$\Delta x = \frac{\Delta a + \Delta b}{2}, \qquad \Delta y = \frac{\Delta a - \Delta b}{2}. \tag{2}$$

Estas relaciones muestran que un desplazamiento sobre un único eje cartesiano requiere la actuación de ambos motores. Para un movimiento exclusivamente en X, ambos desplazamientos tienen igual magnitud y signo. Para un movimiento exclusivamente en Y, tienen igual magnitud y signos opuestos.

En cambio, cuando $\Delta x = \Delta y$, el desplazamiento correspondiente al motor B es nulo. Cuando $\Delta x = -\Delta y$, es nula la actuación del motor A. Estos casos corresponden a diagonales del plano cartesiano y permiten interpretar el acoplamiento del mecanismo.

Los sentidos físicos de giro dependen de la disposición de las correas, de la identificación de los motores y la conexión de sus bobinas. La convención del programa debe comprobarse con movimientos de prueba antes de interpretar las coordenadas como posiciones físicas.

7.2. Conversión de desplazamientos en pulsos

La generación del movimiento requiere expresar las consignas de los motores como cantidades enteras de pulsos. Para ello se utiliza un factor de conversión K_s, expresado en pulsos por milímetro, que depende del ángulo de paso del motor, la subdivisión de pasos seleccionada en el driver y las características de la transmisión.

Para una polea dentada, el factor se calcula mediante:

$$K_s = \frac{N_{\text{rev}}\, m}{z\, p}, \tag{3}$$

donde N_{rev} es la cantidad de pasos completos por revolución, m es el factor de subdivisión de pasos, z es la cantidad de dientes de la polea y p es el paso de la correa.

En la configuración del firmware se consideran motores de $1{,}8^\circ$ por paso, equivalentes a 200 pasos completos por revolución, subdivisión de $1/16$ de paso y poleas de 20 dientes con correas de paso de $2\,\text{mm}$. Con estos parámetros se obtiene:

$$K_s = \frac{200 \times 16}{20 \times 2} = 80\,\text{pulsos/mm}. \tag{4}$$

Este valor corresponde a los parámetros configurados en el programa y debe ser contrastado con los componentes y los ajustes utilizados en el montaje.

A partir del desplazamiento solicitado, las cantidades de pulsos con signo se determinan como:

$$N_A = \text{round}\left[K_s(\Delta x + \Delta y)\right], \qquad N_B = \text{round}\left[K_s(\Delta x - \Delta y)\right]. \tag{5}$$

La operación de redondeo permite obtener valores enteros compatibles con el accionamiento discreto de los motores. El signo de cada resultado establece el sentido de giro mediante la señal DIR, mientras que su valor absoluto determina la cantidad de pulsos que debe generarse en STEP.

En el módulo `stepper`, estas operaciones se realizan a partir de la diferencia entre el destino solicitado y la posición lógica conservada por el firmware. La configuración permite, además, invertir el sentido de X en la transformación cinemática para adecuarlo al montaje.

7.3. Interpolación y sincronización de los motores

La transformación cinemática determina las cantidades totales de pulsos necesarias para alcanzar el destino. Sin embargo, estas cantidades no definen por sí solas la trayectoria seguida durante el desplazamiento. También debe establecerse cómo se distribuyen los pulsos de ambos motores a lo largo del segmento.

Si todos los pasos de un motor fueran ejecutados antes que los del otro, podría alcanzarse el mismo destino ideal mediante una trayectoria diferente de la recta solicitada. Por ello, los accionamientos deben coordinarse durante la ejecución.

En el proyecto se utiliza una interpolación discreta basada en el algoritmo de Bresenham. Se identifica como accionamiento principal al motor que requiere la mayor cantidad de pulsos:

$$N_{\text{mayor}} = \text{máx}(|N_A|, |N_B|), \qquad N_{\text{menor}} = \text{mín}(|N_A|, |N_B|). \tag{6}$$

En cada evento de temporización se genera un pulso para el motor principal. La generación del pulso del motor secundario se decide mediante un acumulador de error, que permite distribuir sus pasos proporcionalmente durante el recorrido.

El acumulador se inicializa con la mitad entera de N_{mayor}. En cada evento se resta N_{menor}; cuando el resultado es negativo, se genera también un pulso para el motor secundario y se suma N_{mayor} al acumulador. De esta manera se conserva la proporción requerida sin efectuar una división en cada decisión de paso.

Por ejemplo, para un desplazamiento de $\Delta x = 10\,\text{mm}$ y $\Delta y = 5\,\text{mm}$, con el factor configurado de $80\,\text{pulsos/mm}$, se obtiene:

$$N_A = 1200, \qquad N_B = 400. \tag{7}$$

El motor A constituye el accionamiento principal y recibe un pulso en cada evento. Los 400 pulsos del motor B se distribuyen durante el segmento, aproximadamente uno cada tres eventos. La relación entre ambos accionamientos permite aproximar la trayectoria rectilínea mediante movimientos discretos.

Para visualizar la distribución sin una secuencia de 1200 eventos, la Tabla 6 utiliza $N_{\text{mayor}} = 12$ y $N_{\text{menor}} = 4$, con la misma proporción 3:1 del ejemplo. El error inicial es 6; la condición es estrictamente negativa.

Cuadro 6. Secuencia ilustrativa de Bresenham calculada con la regla del firmware. Un 1 indica un pulso.

Evento	1	2	3	4	5	6	7	8	9	10	11	12
Principal	1	1	1	1	1	1	1	1	1	1	1	1
Secundario	0	1	0	0	1	0	0	1	0	0	1	0

Cuando ambos motores requieren igual cantidad de pulsos, se acciona cada uno en todos los eventos. Cuando uno de ellos no requiere desplazamiento, solo se generan pulsos para el otro.

La coordinación se realiza desde una misma rutina de interrupción de TIM2. De esta manera, las decisiones de paso de ambos motores comparten una referencia temporal y un contador de avance del segmento.

7.4. Temporización y perfil de velocidad

El parámetro F de la instrucción G-code representa el avance a lo largo de la trayectoria, expresado en milímetros por minuto. Para su utilización en los cálculos de movimiento se convierte a milímetros por segundo:

$$v = \frac{F}{60}. \tag{8}$$

La frecuencia de pasos del accionamiento principal debe ajustarse según la orientación y la longitud del segmento. Una misma frecuencia de motor no produce necesariamente la misma velocidad cartesiana en todos los movimientos CoreXY.

La longitud del desplazamiento solicitado se obtiene mediante:

$$L = \sqrt{(\Delta x)^2 + (\Delta y)^2}. \tag{9}$$

Para una velocidad cartesiana nominal v, la frecuencia de eventos del motor principal resulta:

$$f_{\text{mayor}} = v\,\frac{N_{\text{mayor}}}{L}. \tag{10}$$

En el firmware esta conversión se implementa mediante un factor que relaciona los pulsos del motor principal con la longitud cartesiana. Así, el avance solicitado se interpreta respecto del recorrido del cabezal y no de uno de los accionamientos por separado.

TIM2 se configura con una base temporal nominal de $1\,\mu$s. Durante la ejecución, su período se actualiza para establecer el intervalo hasta el siguiente evento de paso. La rutina de interrupción genera los pulsos, actualiza el contador de avance y calcula la temporización siguiente.

Para evitar cambios instantáneos de velocidad se implementan fases de aceleración, velocidad de crucero y desaceleración. Durante las rampas, la velocidad de pasos se calcula a partir de una relación de la forma:

$$v_p^2 = 2a_p n, \tag{11}$$

donde v_p es la velocidad expresada en pulsos por segundo, a_p es la aceleración en pulsos por segundo cuadrado y n representa la cantidad de eventos recorridos durante la aceleración o restantes durante la desaceleración.

El período entre eventos se obtiene como el inverso de la velocidad de pasos. También se establece un período mínimo configurado para limitar la frecuencia máxima de generación.

Las extensiones de las rampas se ajustan cuando el segmento no dispone del recorrido previsto para ambas fases. Asimismo, las condiciones de entrada y salida a velocidad de crucero pueden ser modificadas por el mecanismo de *lookahead*, descrito en la Subsección 6.5.

7.5. Ejecución del barrido de grabado

En el grabado por barrido, la imagen es representada mediante una secuencia de recorridos lineales. Durante cada línea se ejecutan segmentos consecutivos, con consignas de accionamiento del láser asociadas a la información que debe reproducirse sobre el material.

Al finalizar una línea se realiza un desplazamiento transversal para alcanzar la siguiente. Según la configuración de generación de trayectorias, el recorrido puede alternar su sentido entre líneas o incluir un traslado para comenzar nuevamente desde el mismo lado.

Las consignas cartesianas de cada segmento se transforman mediante la cinemática CoreXY y sus pulsos se distribuyen mediante la interpolación descrita. Para las líneas paralelas a X, ambos motores reciben cantidades iguales de pulsos con el mismo signo; para el avance transversal sobre Y, las cantidades son iguales y sus signos opuestos.

El valor S puede variar entre segmentos de una misma línea. Por lo tanto, la reproducción del grabado requiere coordinar el desplazamiento con la consigna del láser, que puede corresponder a una salida nula o a diferentes ciclos de trabajo del PWM.

En el modo M4, la implementación utiliza además la relación entre la velocidad calculada por el perfil y la velocidad de crucero para ajustar el accionamiento del láser. Esta velocidad es determinada por el programa; no corresponde a una medición del movimiento físico.

7.6. Movimiento independiente del eje Z

El desplazamiento del eje Z se implementa en el módulo `z_axis` y se temporiza mediante TIM4. Su consigna se convierte en pulsos considerando los parámetros del motor, el micropaso y el avance del husillo.

Los dos motores de Z se encuentran documentados como conectados en paralelo a un mismo driver, por lo que reciben una única secuencia de accionamiento y no son controlados individualmente por el firmware.

El eje Z no participa de la interpolación CoreXY ni del almacenamiento de segmentos X–Y. Su ejecución se realiza de manera independiente para ajustar la posición vertical del cabezal. Por ello, la implementación no debe interpretarse como una interpolación simultánea de tres ejes.

7.7. Resolución y exactitud del posicionamiento

La conversión a cantidades enteras de pulsos introduce una discretización del movimiento. La subdivisión de pasos y la relación de transmisión determinan la resolución nominal de las consignas, pero no establecen por sí solas la exactitud del desplazamiento físico.

La coordinación de los pulsos permite conservar la relación de movimiento requerida por la cinemática CoreXY y aproximar la trayectoria programada. La exactitud resultante depende también de la correspondencia entre los parámetros configurados y el montaje real, de las holguras, de la elasticidad y tensión de las correas y de la ejecución de los pasos sin pérdidas.

El posicionamiento se realiza en lazo abierto. Las posiciones conservadas por el firmware son valores lógicos y no mediciones obtenidas mediante sensores de posición. En la implementación revisada, la posición lógica X–Y se actualiza al destino antes de finalizar el desplazamiento.

La exactitud y la repetibilidad deben evaluarse mediante ensayos de la máquina. El conteo lógico no comprueba el desplazamiento físico y, tras una detención, no debe confundirse con una posición medida.

8. Referenciación, Límites y Control del Láser

9. Ensayos y Resultados

10. Limitaciones y Conclusiones

Referencias

[1] Javierarnedo. Core xy: diseño estructural de referencia. Thingiverse, objeto 2818865. https://www.thingiverse.com/thing:2818865. Consulta: 12 de octubre de 2026.

[2] LaserGRBL. Configuration. https://lasergrbl.com/configuration/. Consulta: 12 de octubre de 2026.

[3] LaserGRBL. Raster image import. https://lasergrbl.com/usage/raster-image-import/. Consulta: 12 de octubre de 2026.

[4] Ilan E. Moyer. Corexy: Principle of operation. https://www.corexy.com/theory.html. Consulta: 12 de octubre de 2026.

[5] Nicoop. Cnc_laser_micro: código del firmware. Repositorio GitHub. Versión revisada: commit 688d31a476e18220cf4f35ae20dfb52ec11ff47b. https://github.com/Nicoop/CNC_Laser_micro/tree/688d31a476e18220cf4f35ae20dfb52ec11ff47b.

[6] Proyecto Grbl. Grbl v1.1 interface. https://github.com/gnea/grbl/wiki/Grbl-v1.1-Interface. Consulta: 12 de octubre de 2026.

[7] STMicroelectronics. Um1724: Stm32 nucleo-64 boards (mb1136). Manual de usuario. https://www.st.com/resource/en/user_manual/DM00105823-.pdf. Consulta: 12 de octubre de 2026.